idat

CURSO

Devops: Fundamentos

PARCIAL

Parcial (Linux / Python)

DOCENTE

Cespedes Longobardi Jorge Alfonso

INTEGRANTES

Jimenez Rojas, Jonathan Jose

Revilla Huapaya, Javier Alberto

Talledo Ceverino, Andy Steve

Lima – Perú

2026

CONTENIDO

1. Introducción 3
2. Arquitectura de la solución 4
3. Linux: comandos, automatización y permisos 5
4. Python: procesamiento de inventarios 13
5. Git: control de versiones 20
6. Integración REST API sobre HTTPS 26
7. Problemas encontrados y soluciones 29
8. Sustentación de decisiones técnicas 31
9. Conclusiones 32
Anexo: guía de ejecución 33

1. Introducción

1.1 Contexto

Como integrantes del equipo de Administración de Red, gestionamos dispositivos y servicios distribuidos en varias sedes de la organización. La información de cada sede llega en un formato distinto y además debe contrastarse con un servicio que expone el inventario mediante una API. Hacerlo de forma manual es lento y propenso a errores.

1.2 Objetivo

Desarrollar un proyecto de automatización que permita consultar el inventario mediante una API REST sobre HTTPS, procesar y analizar los datos con Python en varios formatos, versionar el desarrollo con Git y desplegarlo, probarlo y depurarlo en un entorno Linux real.

1.3 Entorno de trabajo

Componente	Detalle
Máquina virtual	DEVASC-LABVM en Oracle VirtualBox (4 vCPU, 8 GB de RAM)
Sistema operativo	Ubuntu 20.04 LTS (64 bits)
Python	3.8.2 con requests 2.22 y PyYAML 5.3
Git	2.25.1
Herramientas	Visual Studio Code, OpenSSL, curl, cron
Ruta del proyecto	~/labs/devnet-src/python/devops_parcial

Tabla 1. Entorno de desarrollo y pruebas.

2. Arquitectura de la solución

El proyecto evolucionó en dos versiones. La primera (v1.0.0) consistió en un script con el inventario definido en el código, validaciones básicas y un CRUD por consola. La segunda (v2.0.x) reorganiza el código en módulos con una única responsabilidad cada uno, lo que permite probarlos por separado y reutilizarlos. Las reglas de negocio se definen una sola vez (modelo.py) y las usan los lectores de archivos, el CRUD, el cliente de la API y el servidor de pruebas.

```
devops_parcial/
├── main.py          CLI: validar, reporte, convertir, api,
│                    diagnostico, api-externa, menu
├── inventario_red/  modelo, formatos, analisis, cliente_api,
│                    crud, registro, config
├── api_simulada/    servidor_https.py (API REST HTTPS)
├── datos/           Lima .json · Arequipa .yaml
│   │                Trujillo .csv · Cusco .xml
│   └── pruebas/     archivos con errores intencionales
├── scripts/         setup_entorno, generar_certificado,
│                    api, auditar_permisos (.sh)
├── tests/           pruebas automatizadas (unittest)
└── legacy/          inventario.py, inventario_v2.py
```

Aspecto	Versión 1 (v1.0.0)	Versión 2 (v2.0.2)
Linux	Comandos ejecutados manualmente	Scripts reutilizables, grupo de trabajo, permisos y tarea programada
Git	Rama master y una rama de trabajo	Ramas main, develop, feature y hotfix; versiones etiquetadas
Python	Lista fija en el código, CRUD en memoria	Paquete modular, cuatro formatos de datos, persistencia y pruebas
REST API	—	Cliente HTTPS con token, reintentos y diagnóstico de errores

Tabla 2. Evolución del proyecto entre versiones.

3. Linux: comandos, automatización y permisos

3.1 Comandos fundamentales

En la primera etapa preparamos el entorno y practicamos los comandos base de navegación, permisos y gestión de usuarios.

- Nos ubicamos en la carpeta de trabajo de Python (retrocedemos con cd .. si es necesario).

```
cd labs/devnet-src/python
```

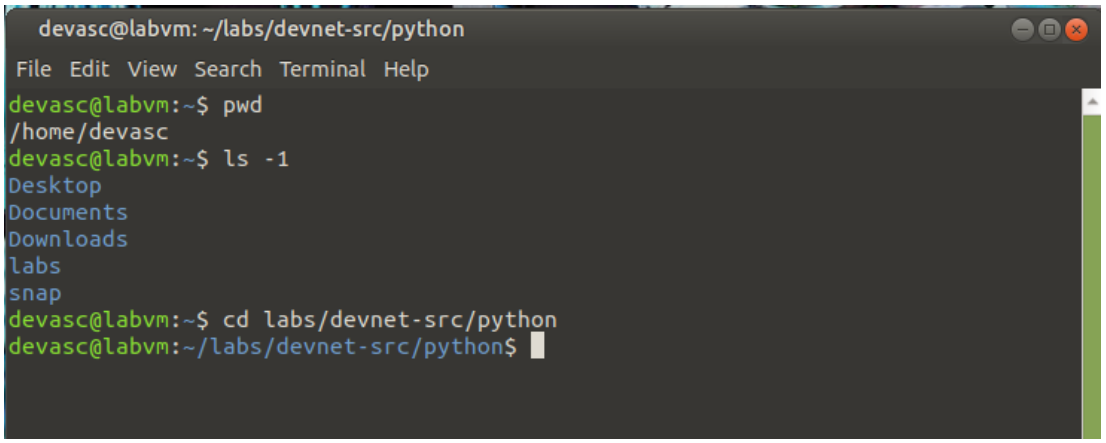


Figura 1. Ubicación en labs/devnet-src/python.

- Creamos la carpeta del proyecto.

```
mkdir devops_parcial
```

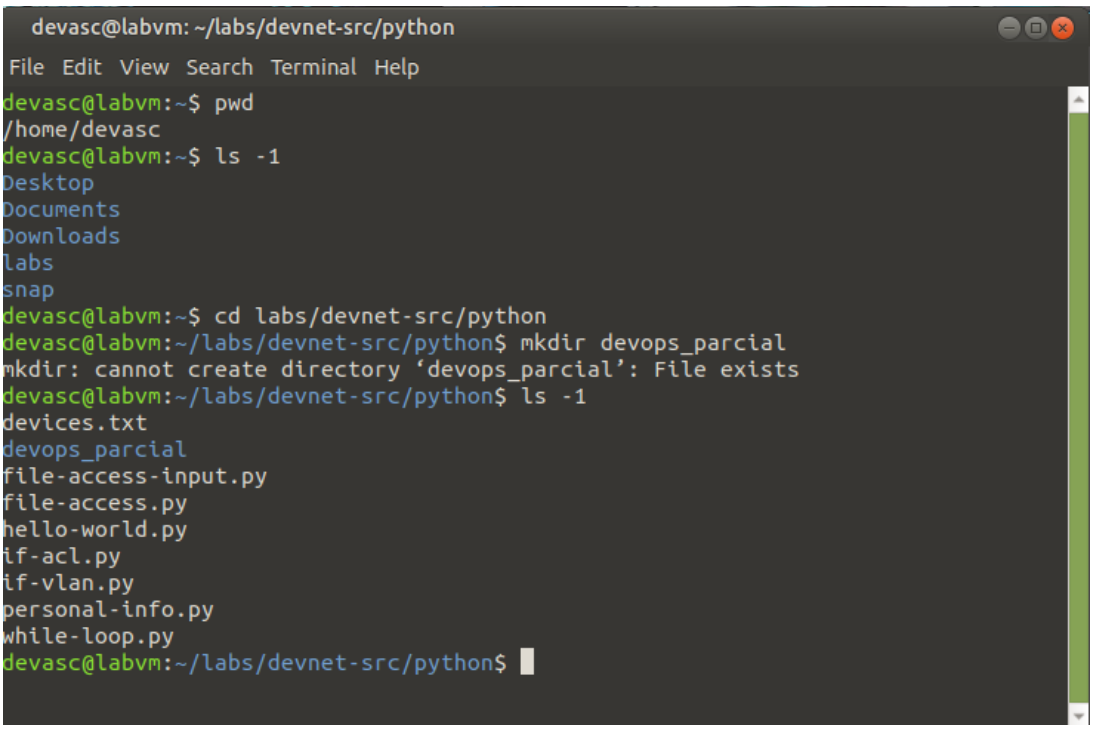


Figura 2. Creación de la carpeta devops_parcial.

- Consultamos los permisos actuales del archivo.

```
ls -l devops_parcial/inventario.py
```

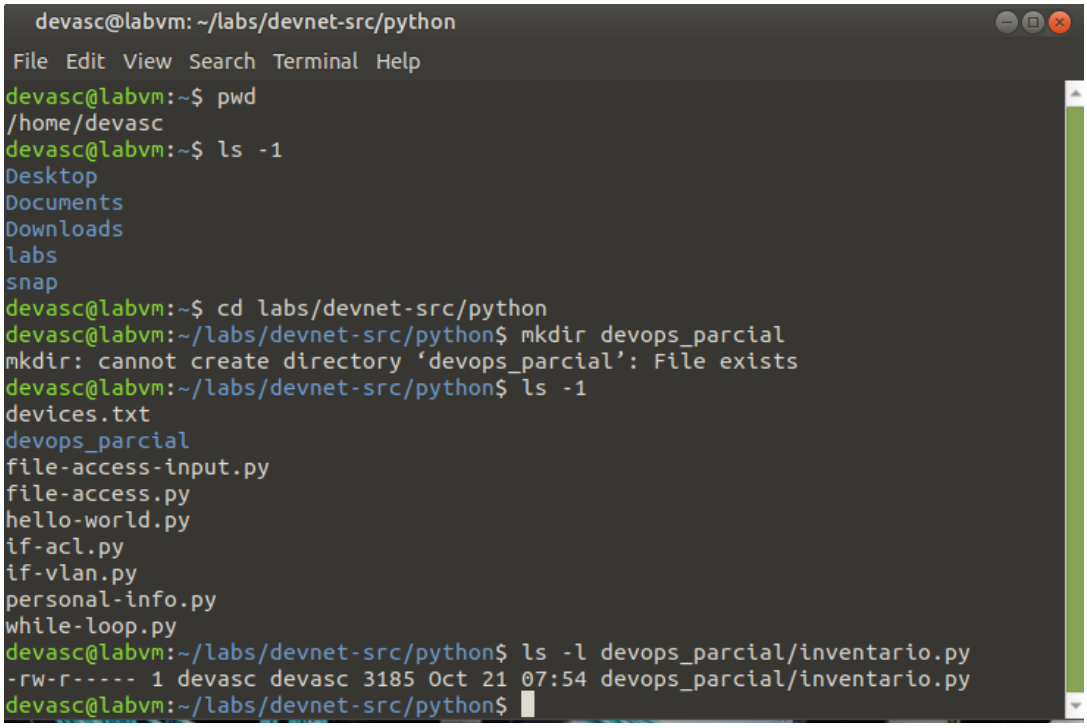


Figura 3. Permisos iniciales de inventario.py.

- Cambiamos el permiso con chmod 640: el propietario lee y escribe, el grupo solo lee y los demás usuarios no tienen acceso.

```
chmod 640 devops_parcial/inventario.py
```

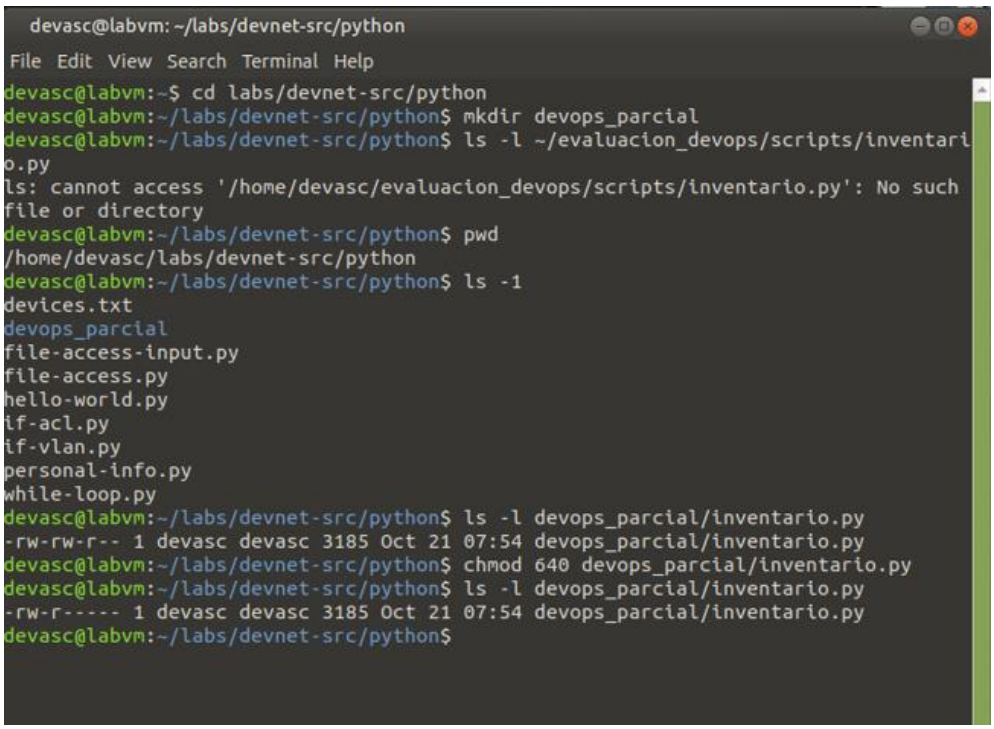


Figura 4. Permisos después de chmod 640.

- Ejecutamos el programa desde la terminal.

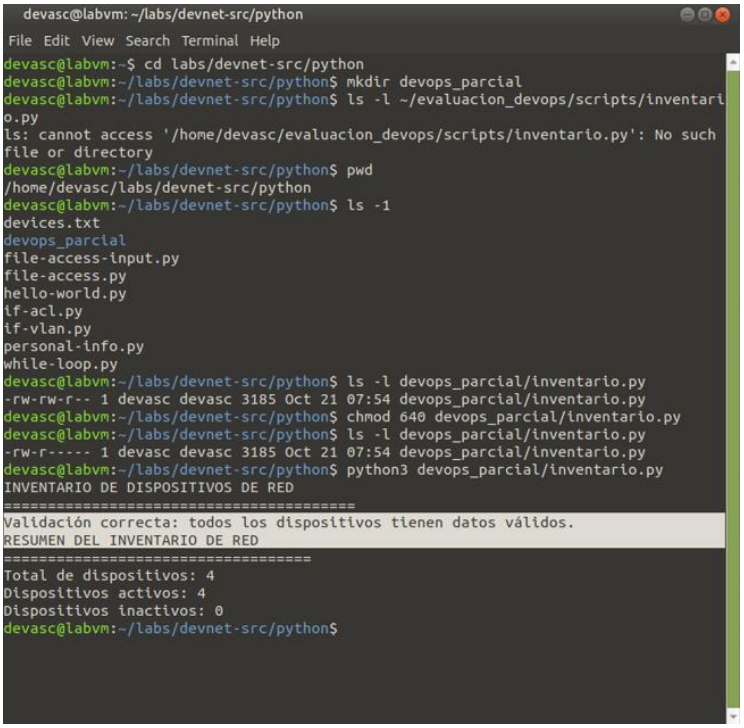


Figura 5. Ejecución del script desde la terminal.

- Consultamos la información detallada del archivo: tamaño, propietario, grupo y permisos.

```
stat devops_parcial/inventario.py
```

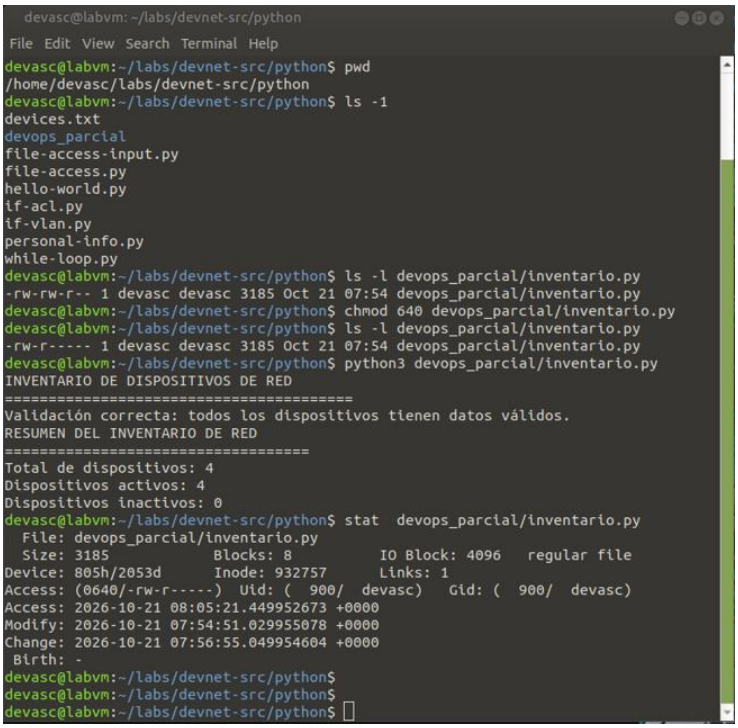


Figura 6. Información del archivo con stat.

- Verificamos qué usuario tiene la sesión iniciada.

```
whoami
```

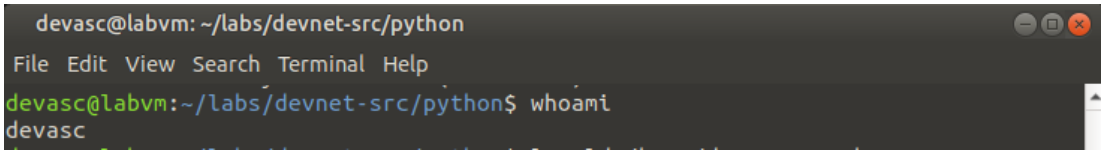


Figura 7. Usuario actual con whoami.

- Mostramos los identificadores y grupos del usuario actual.

```
id
```

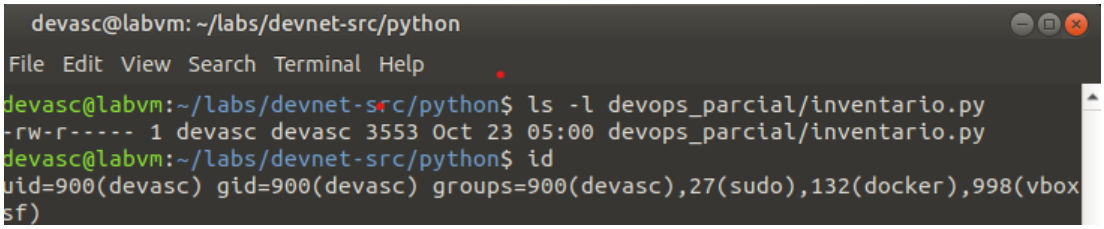


Figura 8. Identificadores y grupos con id.

- Verificamos que el usuario devops_prueba aún no existe.

```
id devops_prueba
```

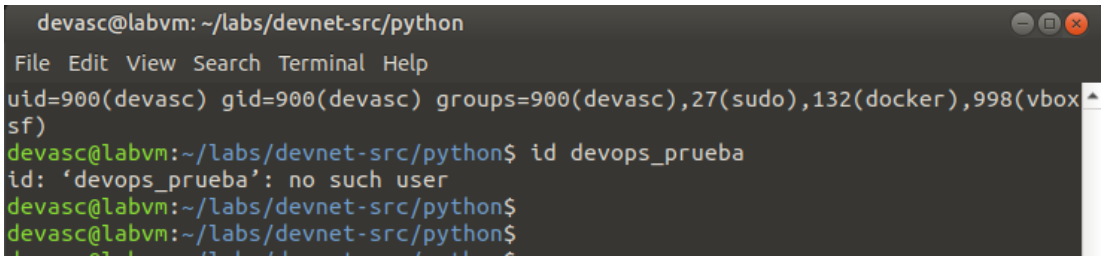


Figura 9. El usuario devops_prueba no existe.

- Creamos el usuario con sudo y lo verificamos nuevamente.

```
sudo useradd -m devops_prueba
```

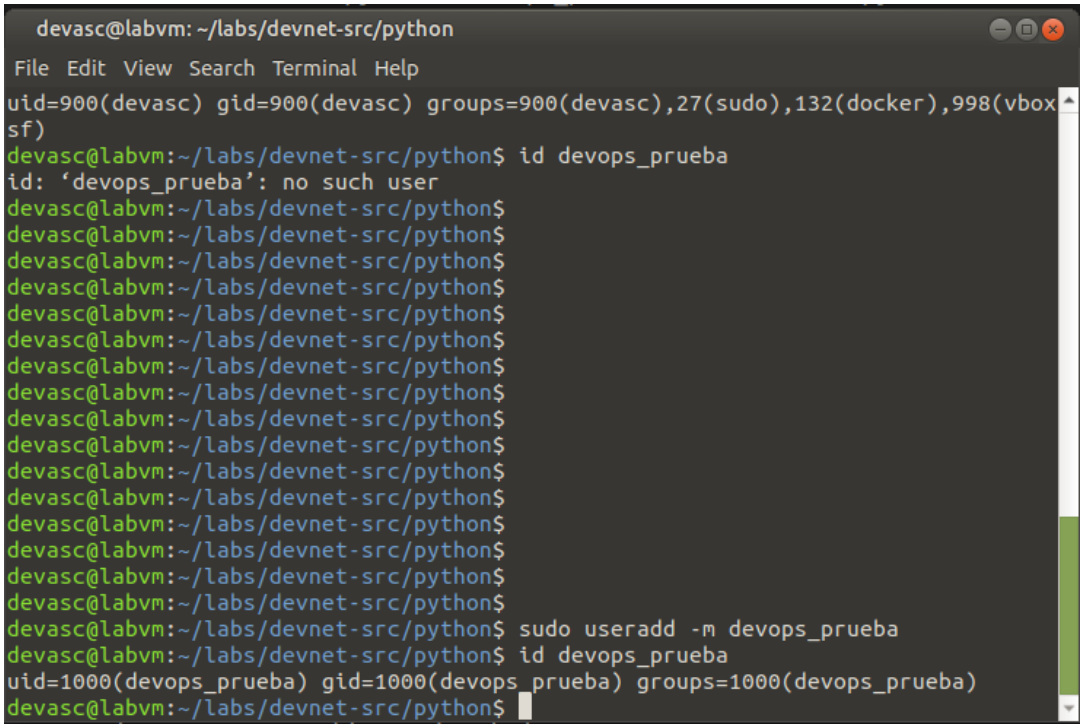


Figura 10. Creación del usuario con useradd.

- Creamos un archivo de prueba independiente.

```
echo "¡Pruebas CHOWN!" >> prueba_permisos.txt
```

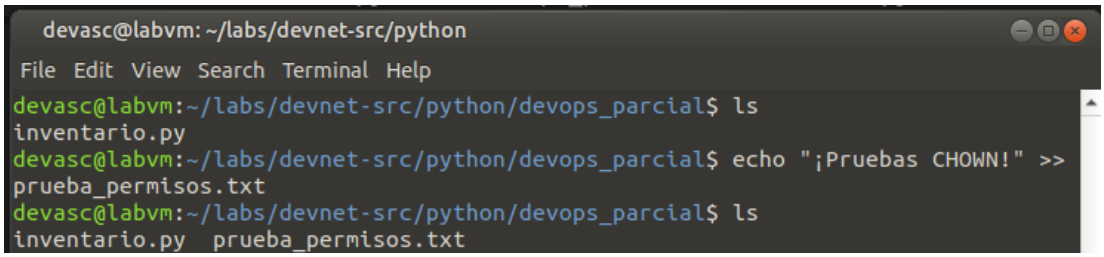


Figura 11. Creación del archivo de prueba.

- Cambiamos el propietario del archivo y lo verificamos.

```
sudo chown devops_prueba devops_parcial/prueba_permisos.txt

ls -l devops_parcial/prueba_permisos.txt
```

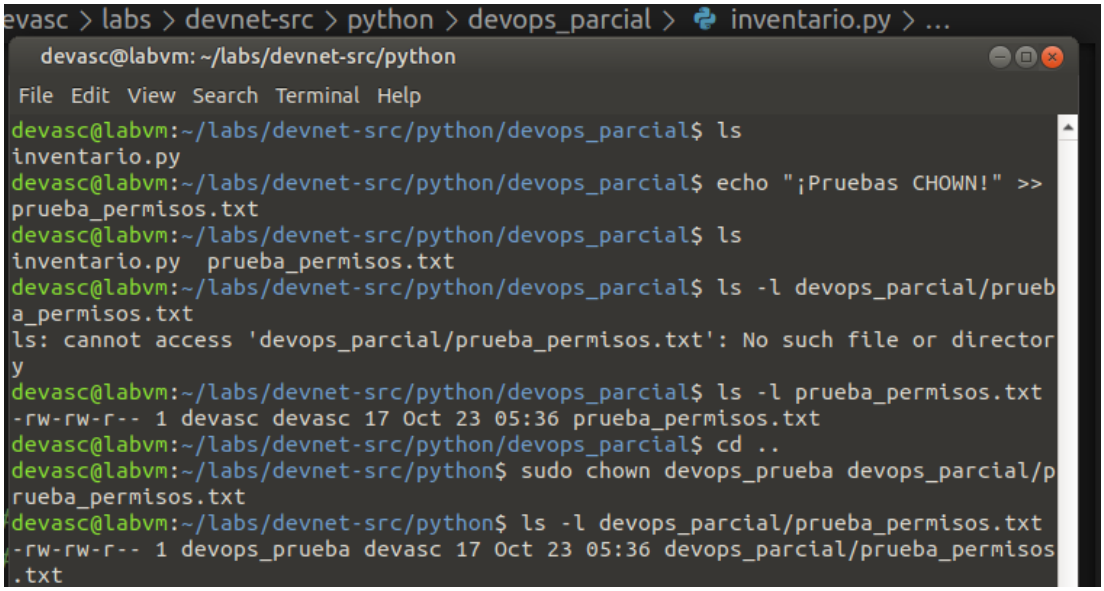


Figura 12. Cambio de propietario con chown.

- Verificamos el contenido del archivo.

```
cat devops_parcial/prueba_permisos.txt
```

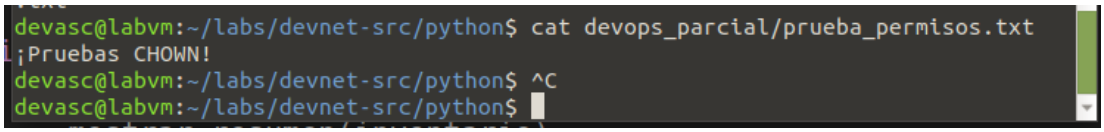


Figura 13. Contenido del archivo de prueba.

3.2 Automatización con scripts Bash

Para que el despliegue sea repetible, convertimos esos comandos en scripts. Todos usan set -euo pipefail, de modo que se detienen ante el primer error, y son idempotentes: pueden ejecutarse varias veces sin duplicar usuarios, grupos ni tareas.

Script	Función	Comandos combinados
setup_entorno.sh	Verifica dependencias, crea la estructura, genera .env con una API key aleatoria, el certificado TLS y aplica permisos	command -v, mkdir -p, openssl rand, umask 077, find -print0 | xargs -0 chmod, stat -c
setup_entorno.sh --equipo	Crea el grupo netops y una cuenta de servicio sin contraseña	getent, groupadd, useradd -m, passwd -l, usermod -aG, chown -R
setup_entorno.sh --cron	Programa el reporte consolidado cada hora	crontab -l | grep -v | crontab -
auditar_permisos.sh	Detecta archivos escribibles por otros, secretos sin 600 y secretos versionados	find -perm -o+w, stat, git ls-files | grep
api.sh	Inicia, detiene y consulta la API en segundo plano	nohup, kill -0, curl --cacert, ss -tlnp

Tabla 3. Scripts de automatización.

3.3 Estrategia de permisos

Aplicamos el principio de mínimo privilegio: el equipo puede consultar el proyecto, pero solo el propietario lo modifica, y los secretos quedan inaccesibles incluso para el grupo.

Elemento	Permiso	Justificación
Directorios del proyecto	750 (rwxr-x---)	El grupo netops puede entrar y leer; otros usuarios no
Código y datos	640 (rw-r-----)	Solo el propietario modifica; el grupo consulta
main.py y scripts .sh	750	Solo los puntos de entrada son ejecutables
.env y servidor.key	600 (rw-------)	Secretos: solo el propietario
logs/ y reportes/	2770 (rwxrws---)	setgid: lo generado dentro hereda el grupo netops
Archivos creados por Python	umask 027	Se crean sin acceso para otros usuarios

Tabla 4. Permisos aplicados.

3.4 Evidencias

```
File  Edit  View  Search  Terminal  Help
[INFO] Proyecto: /home/devasc/labs/devnet-src/python/devops_parcial (propietario: devasc)
[INFO] Verificando dependencias
[ OK ] python3 -> /usr/bin/python3
[ OK ] git -> /usr/bin/git
[ OK ] openssl -> /usr/bin/openssl
[ OK ] curl -> /usr/bin/curl
[INFO] Creando estructura de directorios
[ OK ] logs/ reportes/ certs/
[INFO] Generando .env con una API key aleatoria
[ OK ] .env con permiso 600
[OK] Certificado generado:
subject=C = PE, L = Lima, O = IDAT DevOps, CN = localhost
notAfter=Nov  6 00:17:42 2027 GMT
X509v3 Subject Alternative Name:
    DNS:localhost, IP Address:127.0.0.1
[INFO] Aplicando permisos (minimo privilegio)
[ OK ] Permisos aplicados
[ OK ] Listo

total 88
drwxr-x--- 12 devasc devasc  4096 Nov  6 00:17 .
drwxr-xr-x  3 devasc devasc  4096 Nov  6 00:16 ..
drwxr-x---  3 devasc devasc  4096 Nov  6 00:17 api_simulada
drwxr-x---  2 devasc devasc  4096 Nov  6 00:17 certs
drwxr-x---  3 devasc devasc  4096 Nov  6 00:17 datos
-rw-------  1 devasc devasc   252 Nov  6 00:17 .env
-rw-r-----  1 devasc devasc   353 Nov  6 00:17 .env.example
drwxrwxr-x  8 devasc devasc  4096 Nov  6 00:17 .git
-rw-r-----  1 devasc devasc   240 Nov  6 00:16 .gitignore
drwxr-x---  3 devasc devasc  4096 Nov  6 00:17 inventario_red
drwxr-x---  2 devasc devasc  4096 Nov  6 00:16 legacy
drwxrws---  2 devasc devasc  4096 Nov  6 00:17 logs
```

Figura 14. Ejecución de setup_entorno.sh.

```
File  Edit  View  Search  Terminal  Help
[INFO] Proyecto: /home/devasc/labs/devnet-src/python/devops_parcial (propietario: devasc)
[INFO] Configurando grupo 'netops' y usuario de servicio 'devops_prueba'
[ OK ] Miembros de netops: devops_prueba,devasc
[INFO] Aplicando permisos (minimo privilegio)
[ OK ] Permisos aplicados
[ OK ] Listo
uid=1000(devops_prueba) gid=1001(devops_prueba) groups=1001(devops_prueba),1000(netops)
netops:x:1000:devops_prueba,devasc
<?xml version="1.0" encoding="UTF-8"?>
<inventario sede="Cusco">
  <dispositivo>
cat: .env: Permission denied
```

Figura 15. Grupo netops: devops_prueba lee los datos, pero no puede leer .env.

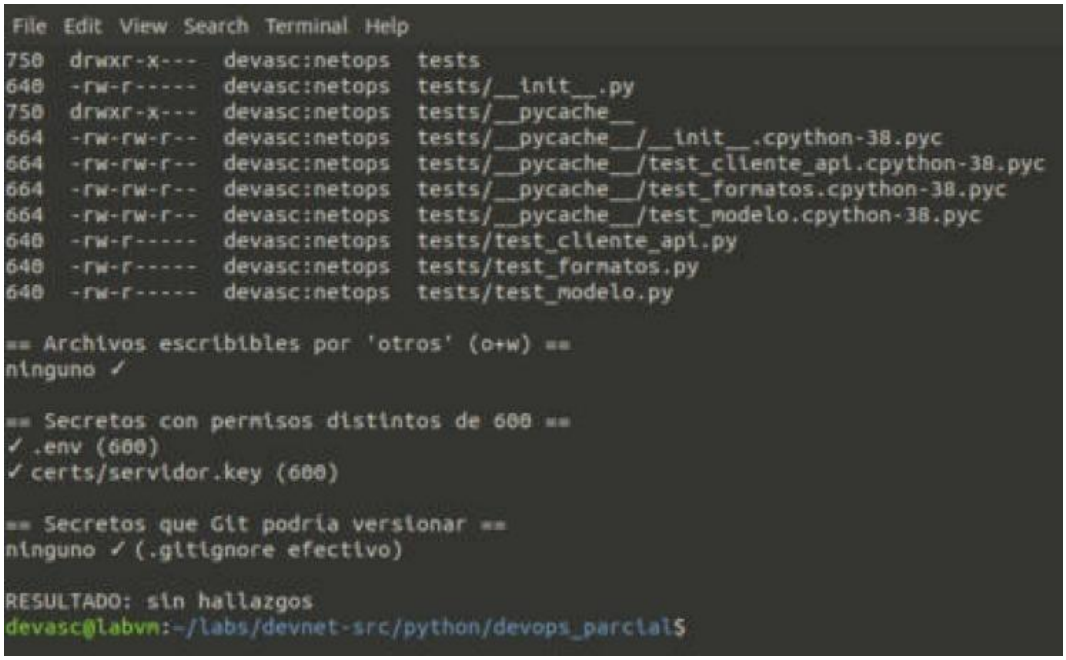


Figura 16. Auditoría de permisos sin hallazgos.

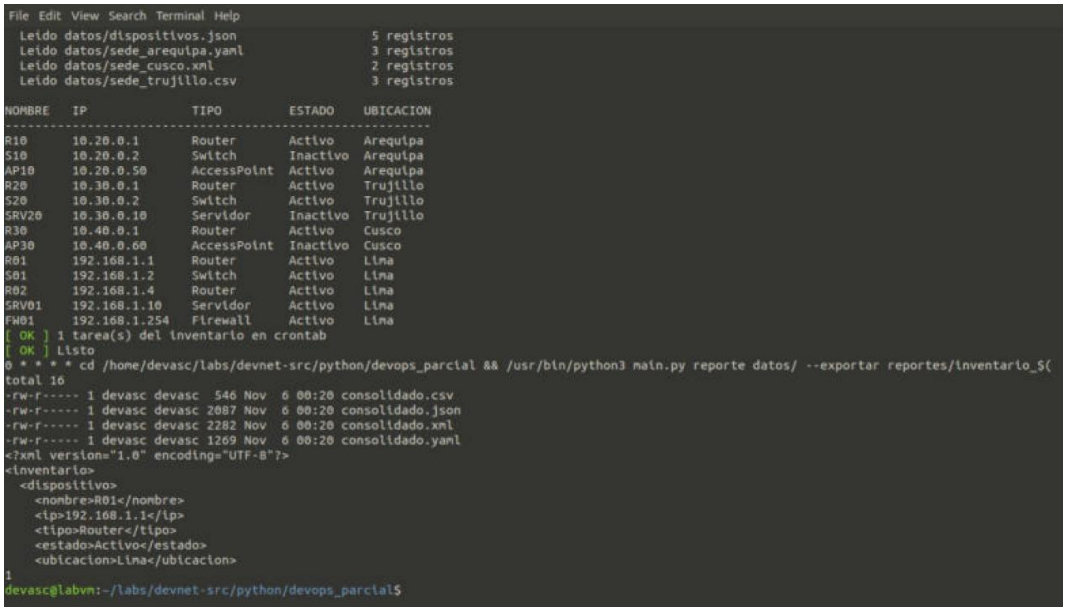


Figura 17. Reporte consolidado, tarea cron y reportes con permiso 640.

4. Python: procesamiento de inventarios

4.1 Primera versión

- Definimos un diccionario con la estructura básica de un dispositivo (VS Code).

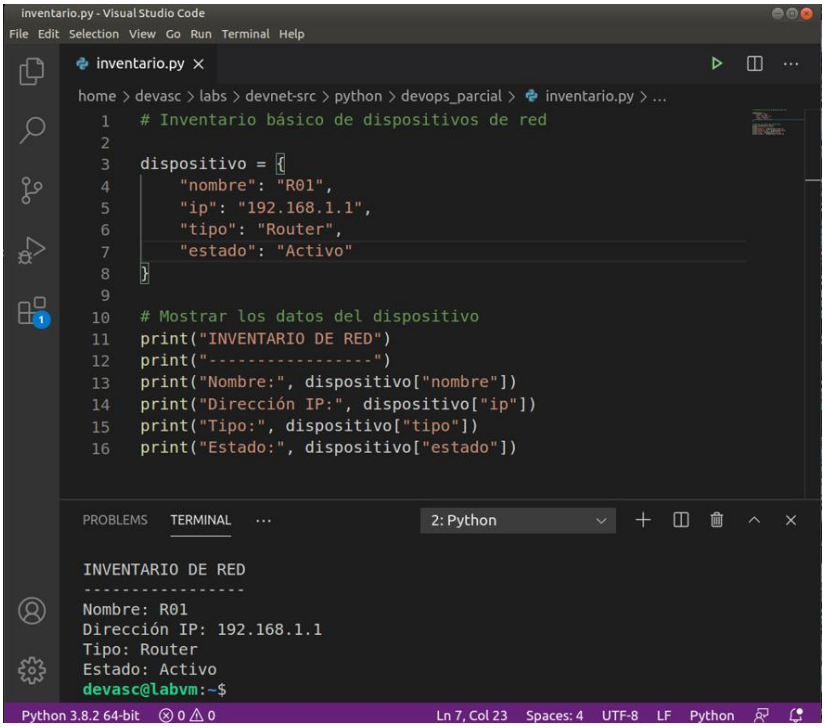


Figura 18. Diccionario inicial de un dispositivo.

- Reemplazamos el diccionario por una lista de dispositivos:

```
inventario = [
    {
        "nombre": "R01",
        "ip": "192.168.1.1",
        "tipo": "Router",
        "estado": "Activo"
    },
    {
        "nombre": "S01",
        "ip": "192.168.1.2",
        "tipo": "Switch",
        "estado": "Activo"
    },
    {
        "nombre": "SRV01",
        "ip": "192.168.1.10",
        "tipo": "Servidor",
        "estado": "Activo"
    },

    {
```

```
        "nombre": "R02",
        "ip": "192.168.1.4",
        "tipo": "Router",
        "estado": "Activo"
    }
]
```

- Recorremos la lista con un bucle for para mostrar los datos automáticamente, sin escribir un print() por dispositivo.

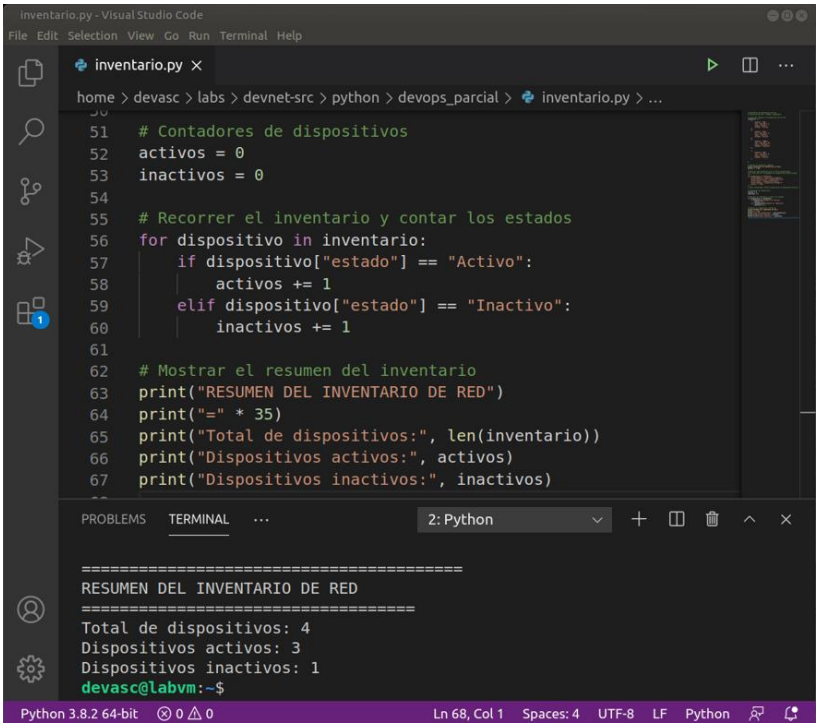


Figura 19. Recorrido del inventario con for.

- Contamos los dispositivos según su estado (Activo / Inactivo).

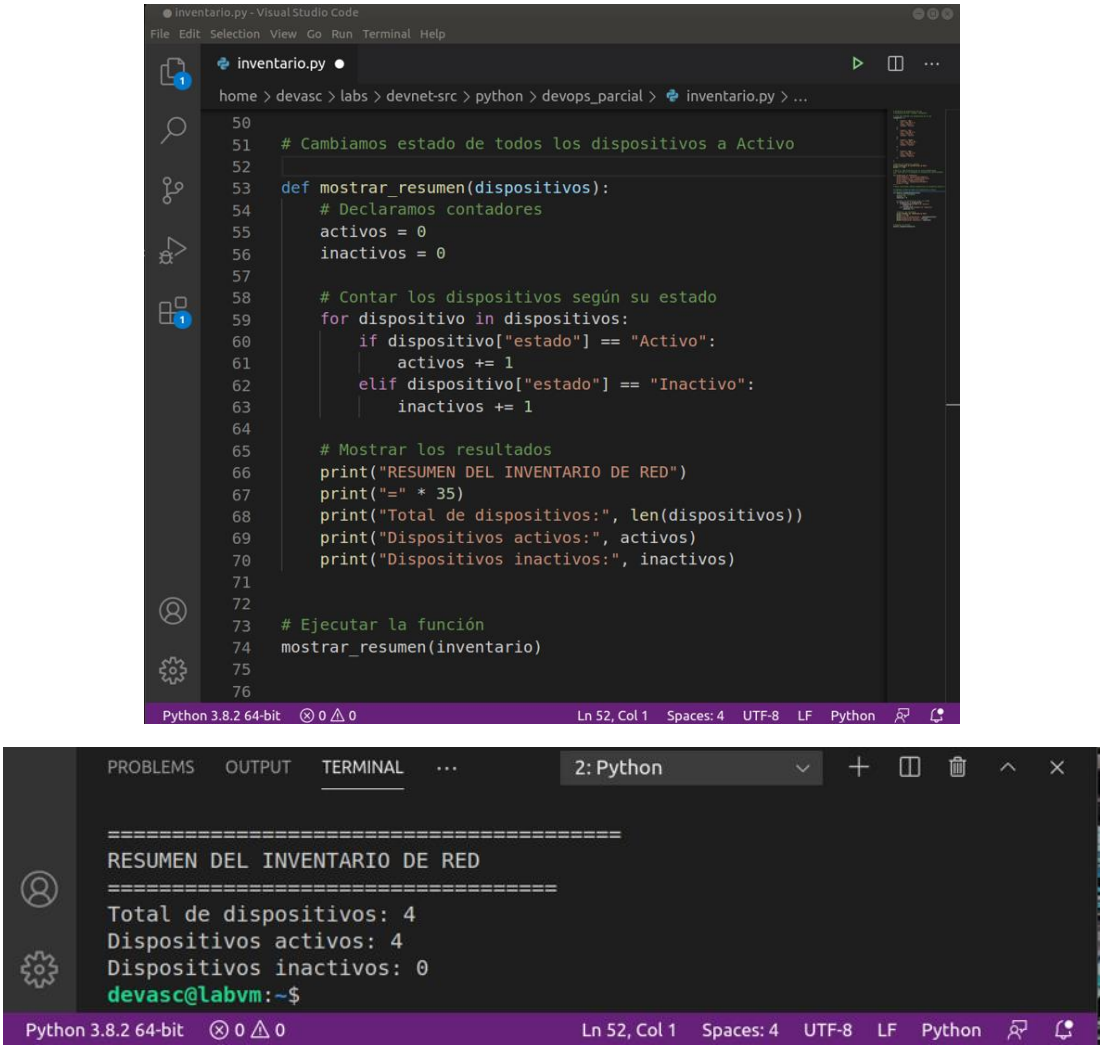


Figura 20. Conteo de dispositivos por estado.

- Encapsulamos el resumen en una función.

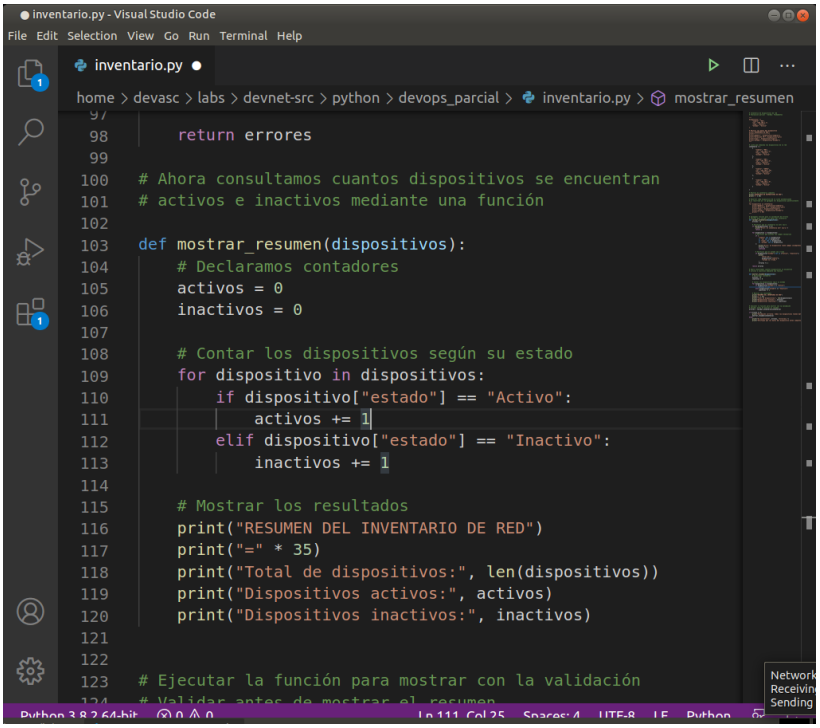


Figura 21. Función de resumen.

- Validamos con una función que ningún dispositivo tenga campos vacíos y que el estado solo sea Activo o Inactivo.

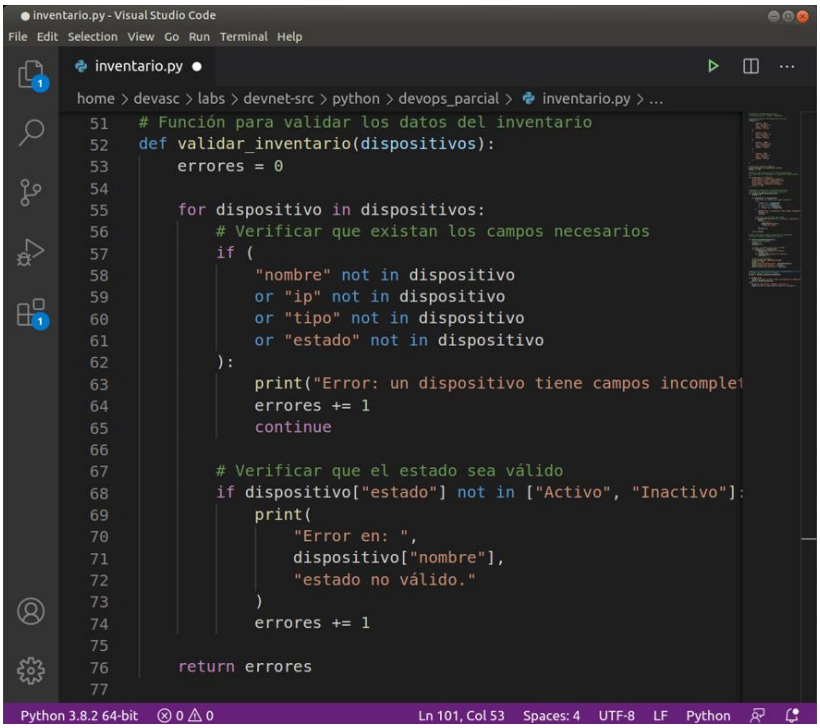
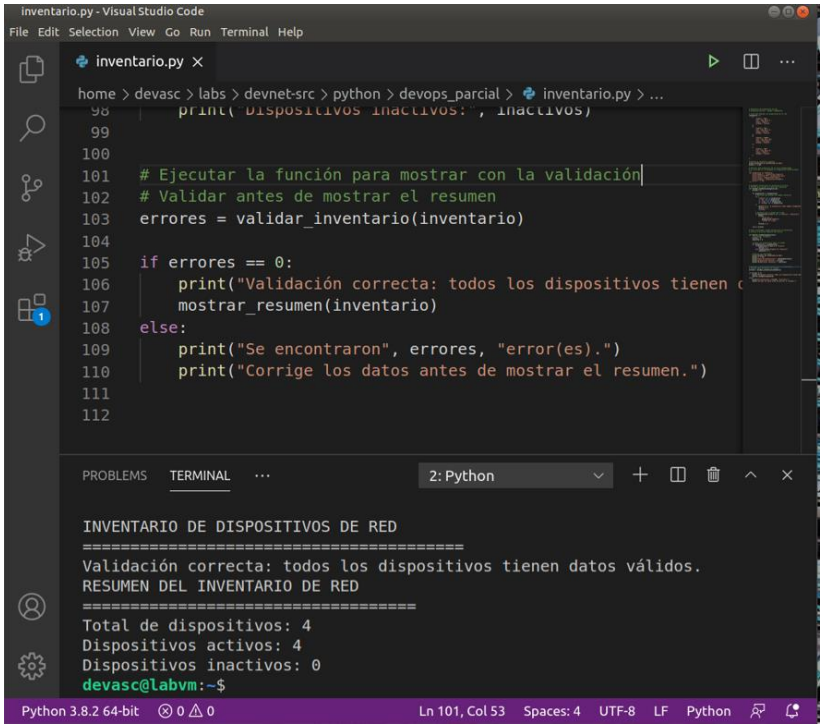


Figura 22. Función de validación.

- Ejecutamos el programa validando los datos antes de mostrar el resumen.



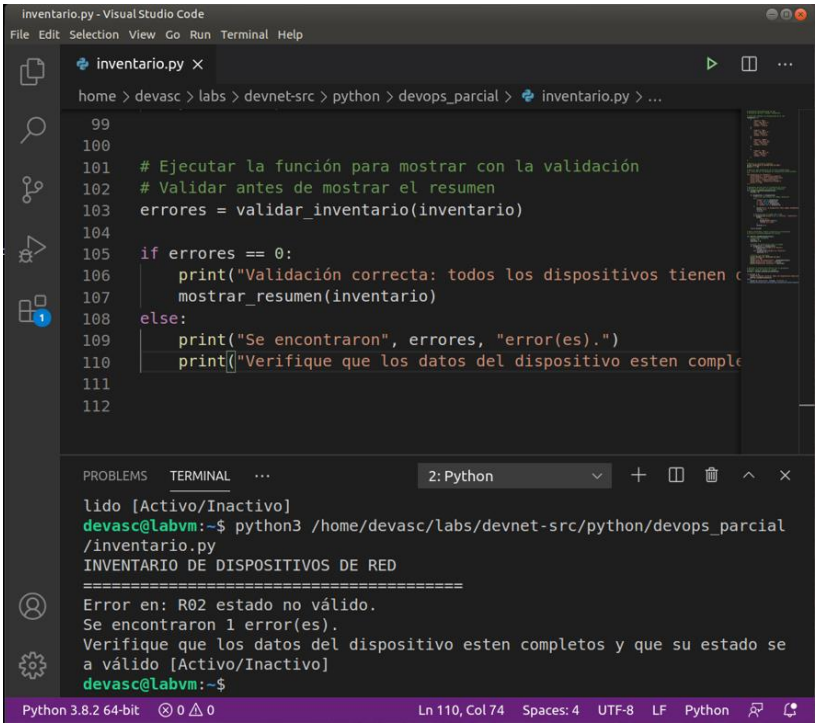


Figura 23. Ejecución con validación previa.

4.2 Versión modular

A partir de la primera versión reorganizamos el código en el paquete inventario_red e incorporamos las siguientes mejoras:

Mejora	Detalle técnico	Beneficio
Modelo único	Clase Dispositivo (@dataclass) que normaliza los datos (r01 → R01) y valida la IP con ipaddress	Una sola regla para todo el sistema

Mejora	Detalle técnico	Beneficio
Validación del conjunto	Detecta nombres e IPs duplicados entre registros y sedes	Evita conflictos de direccionamiento
Lectura multi-formato	Un lector por extensión: json, yaml.safe_load, csv.DictReader, ElementTree	Agregar un formato no afecta al resto
Errores con contexto	Indica archivo, línea y columna (JSON) o columnas faltantes (CSV)	Corrección inmediata
Escritura atómica	Archivo temporal + os.replace, con permiso 640	Un fallo no deja reportes dañados
Análisis	Disponibilidad, agrupación por tipo, sede y subred /24; comparación con la API	Información para decidir
Persistencia del CRUD	Los cambios del menú se guardan en datos/dispositivos.json	No se pierden al cerrar
Interfaz de línea de comandos	argparse con códigos de salida 0, 1 y 2	Uso desde cron o integración continua
Pruebas	20 pruebas con unittest, incluida integración HTTPS	Cambios seguros

Tabla 5. Mejoras de la versión modular.

4.3 Procesamiento de múltiples formatos

Cada sede entrega su inventario en un formato distinto: Lima en JSON, Arequipa en YAML, Trujillo en CSV y Cusco en XML. El comando reporte los consolida, valida y resume, y puede exportar el resultado a cualquiera de los cuatro formatos.

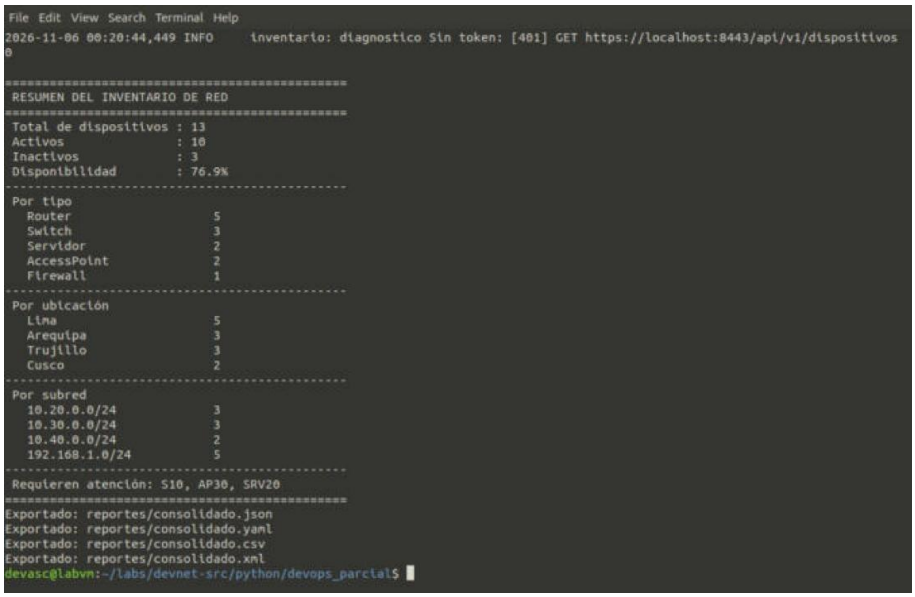


Figura 24. Resumen consolidado de 13 dispositivos de cuatro sedes.

4.4 Validación de datos

Para comprobar las validaciones preparamos archivos con errores intencionales: IP fuera de rango, nombre duplicado, tipo no permitido, campo vacío, estado inválido y JSON mal formado.

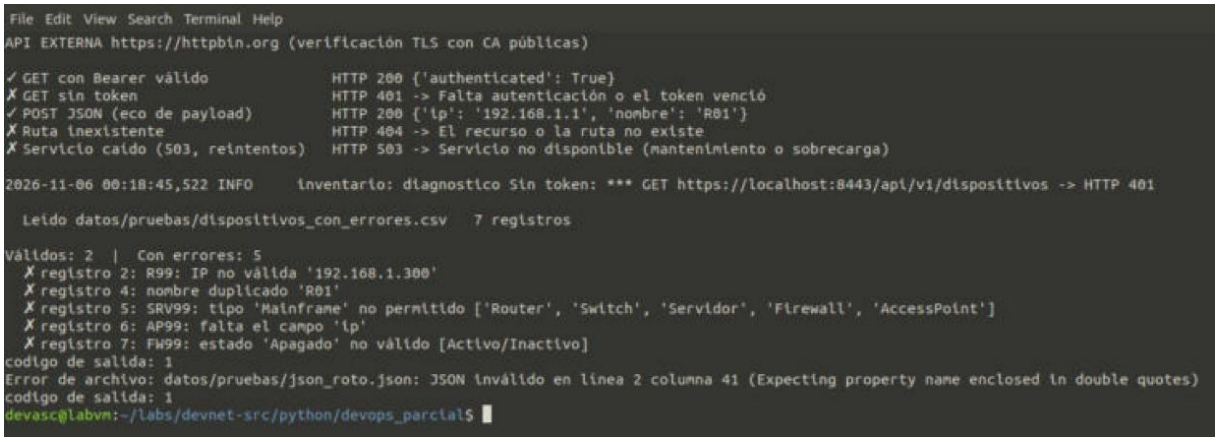


Figura 25. Validación de archivos con errores y consumo de la API externa.

5. Git: control de versiones

5.1 Primeros pasos

- Configuramos el usuario y el correo con git config --global user.name / user.email y los verificamos con git config --list.

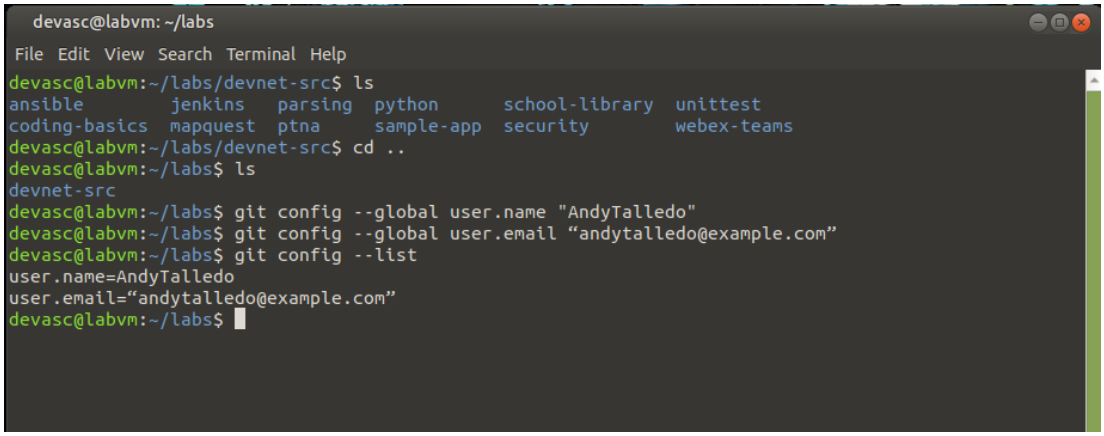


Figura 26. Configuración de identidad en Git.

- Creamos una carpeta, nos ubicamos en ella y la inicializamos como repositorio con git init.



Figura 27. Inicialización del repositorio.

- Copiamos el archivo trabajado a la carpeta con cp, lo preparamos con git add y revisamos el estado con git status.

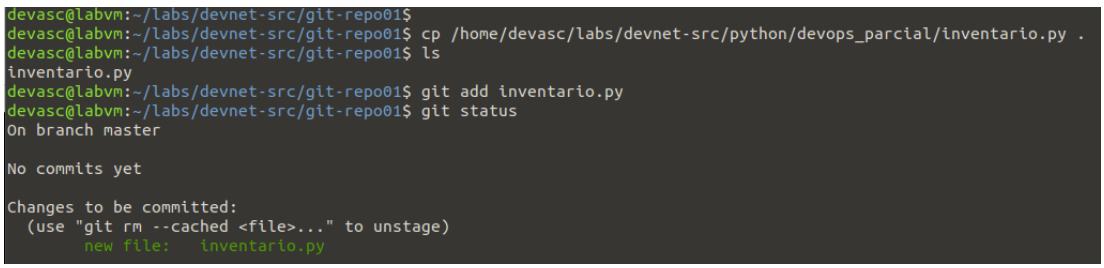


Figura 28. Preparación de cambios con git add.

- Confirmamos los cambios con git commit -m "…" (siempre con un mensaje descriptivo), comprobamos con git status que no quedan pendientes y revisamos el historial con git log.

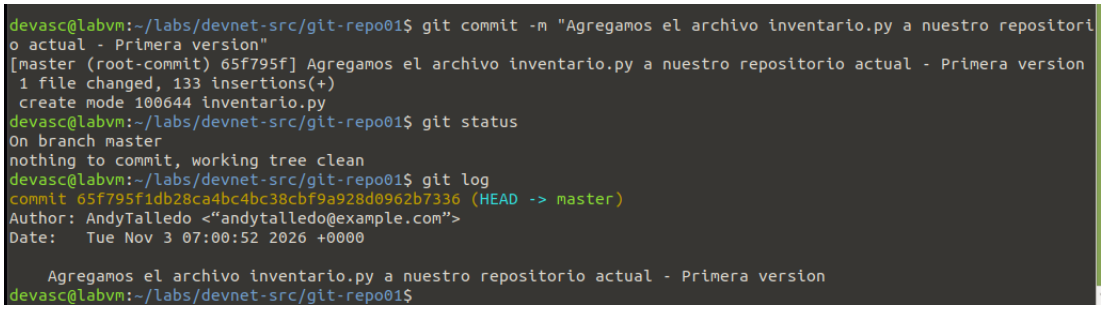


Figura 29. Primer commit e historial.

- Verificamos con git branch que estamos en master. Creamos la rama rama01 (git branch rama01), cambiamos a ella (git checkout rama01) y lo comprobamos.

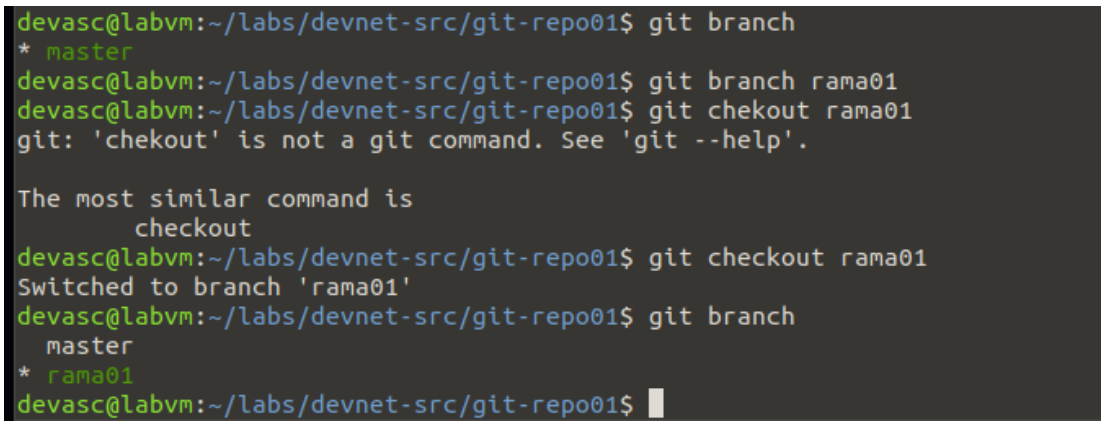


Figura 30. Creación y cambio a rama01.

- Comprobamos con ls que la rama tiene los mismos archivos que master y que inventario.py se ejecuta igual.

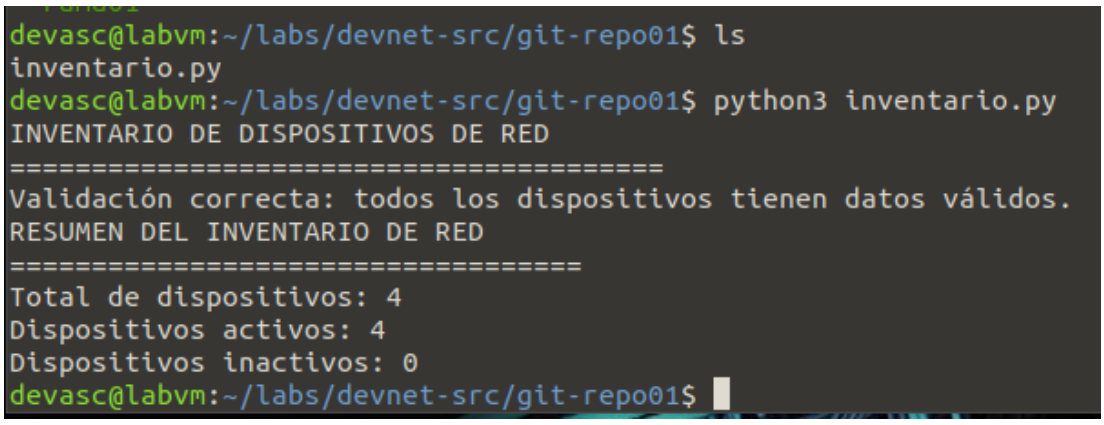


Figura 31. Contenido de rama01.

- Creamos inventario_v2.py con el CRUD de dispositivos y lo ejecutamos en la terminal.

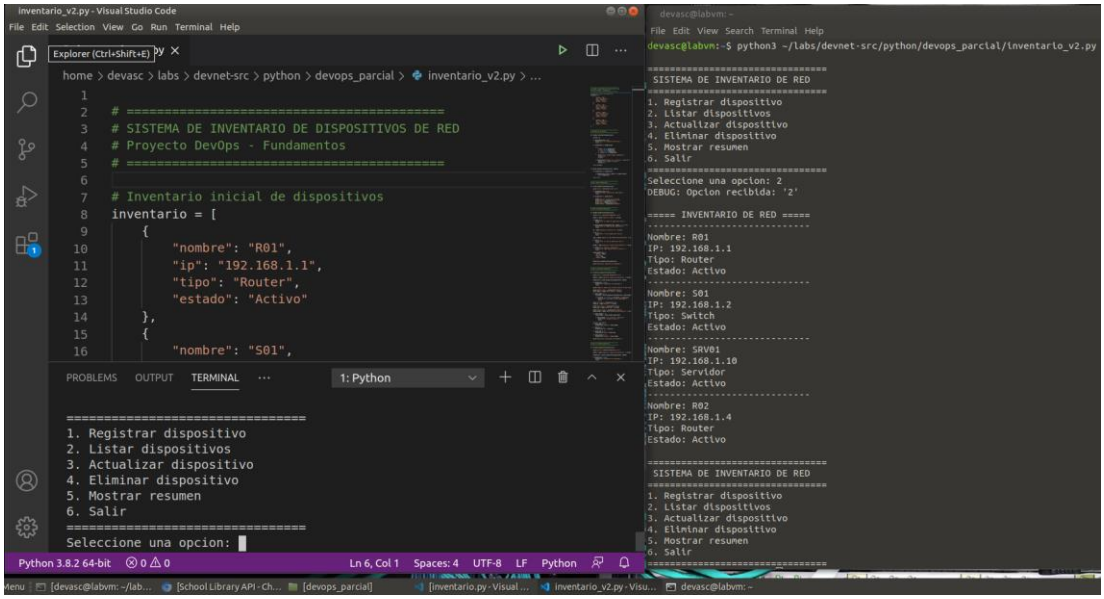


Figura 32. Ejecución de inventario_v2.py (CRUD).

- Agregamos el archivo a rama01 (cp, git add, git commit) y verificamos con git status y git log.

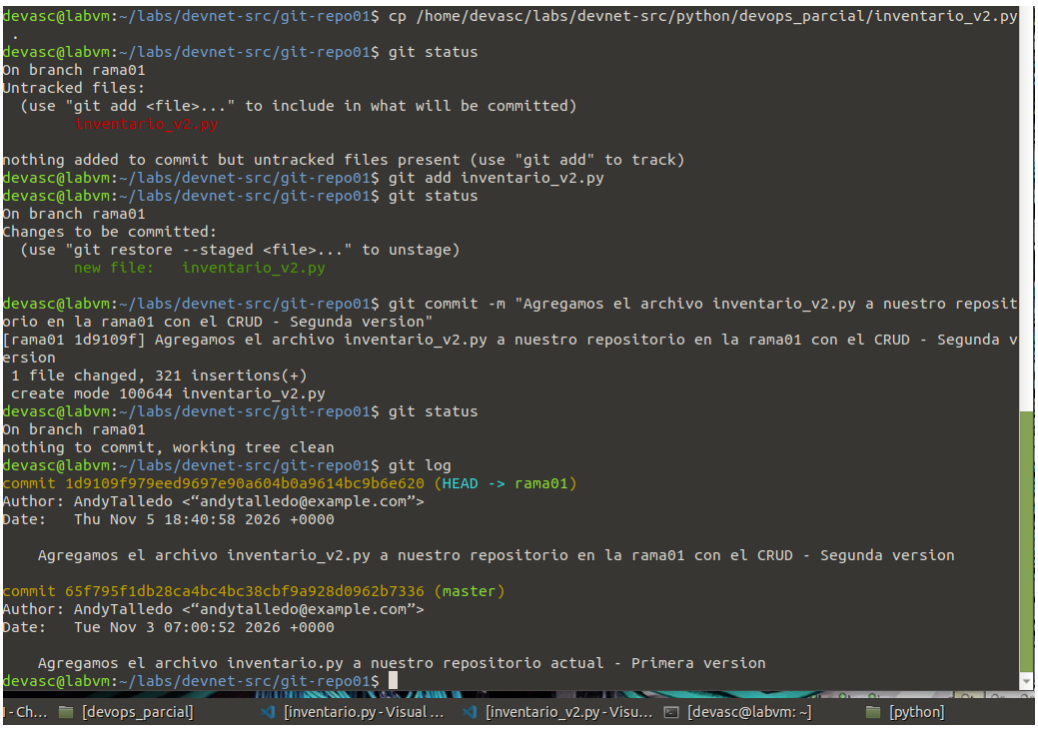


Figura 33. Commit en rama01.

- Volvemos a master, comprobamos con ls que inventario_v2.py aún no existe y ejecutamos git merge rama01. Ahora master tiene ambos archivos y los ejecutamos para confirmar.

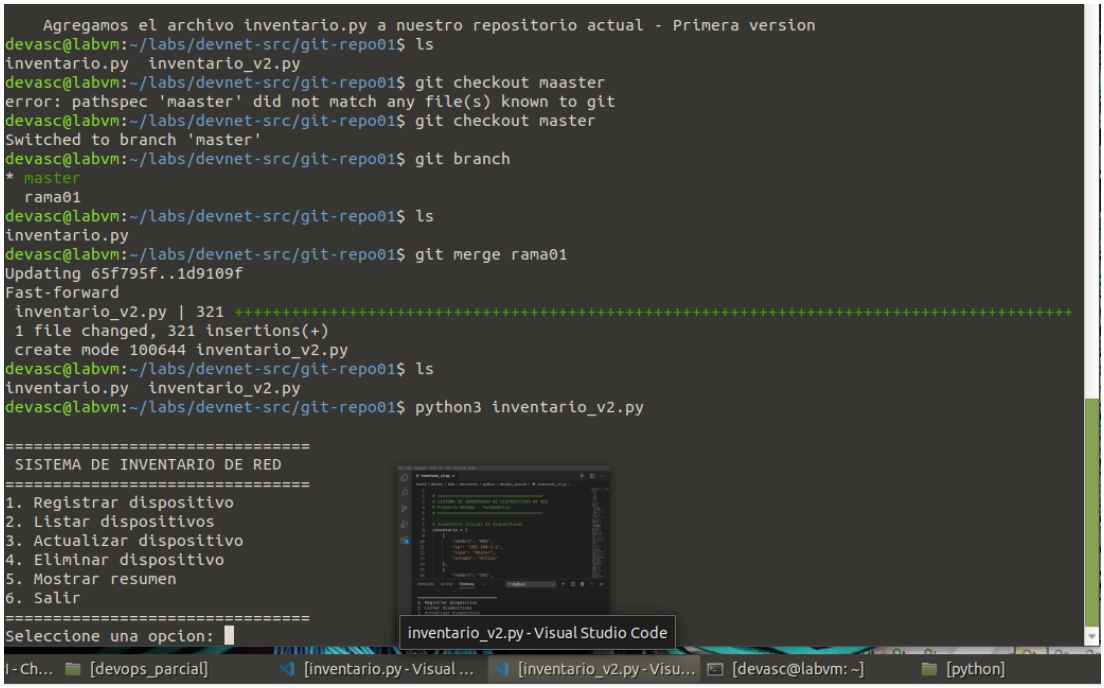


Figura 34. Merge de rama01 en master.

5.2 Flujo de ramas

En la segunda versión adoptamos un flujo de ramas: **main** contiene solo versiones estables y etiquetadas; **develop** integra el trabajo; cada funcionalidad se desarrolla en una rama **feature/** y cada corrección urgente en una rama **hotfix/**. Las integraciones se hacen con merge --no-ff para que el historial muestre qué commits formaron cada funcionalidad.

Práctica	Aplicación
Mensajes de commit	Prefijos feat, fix, test, refactor, ops, docs y chore, con ámbito: feat(api)
.gitignore	Excluye .env, llaves, certificados, logs, reportes y __pycache__
Versiones	v1.0.0 (base y CRUD), v2.0.0 (versión modular), v2.0.1 y v2.0.2 (correcciones)
Pruebas antes de integrar	Cada integración se confirma solo si las pruebas pasan
Correcciones	Cada hotfix se integra en main y luego en develop

Tabla 6. Buenas prácticas de Git.

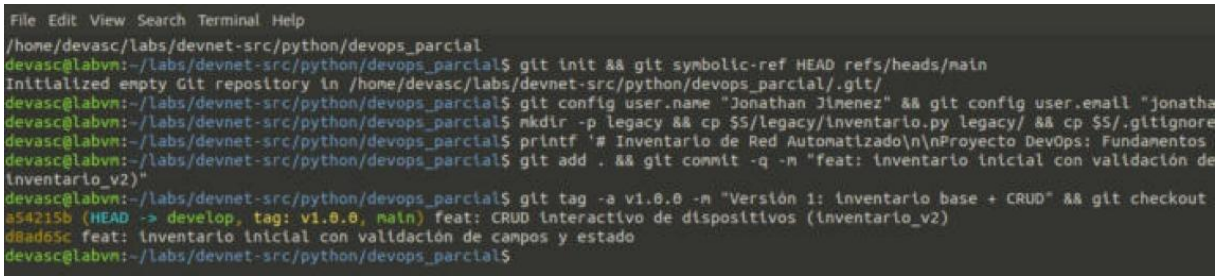


Figura 35. Repositorio con rama main, tag v1.0.0 y rama develop.

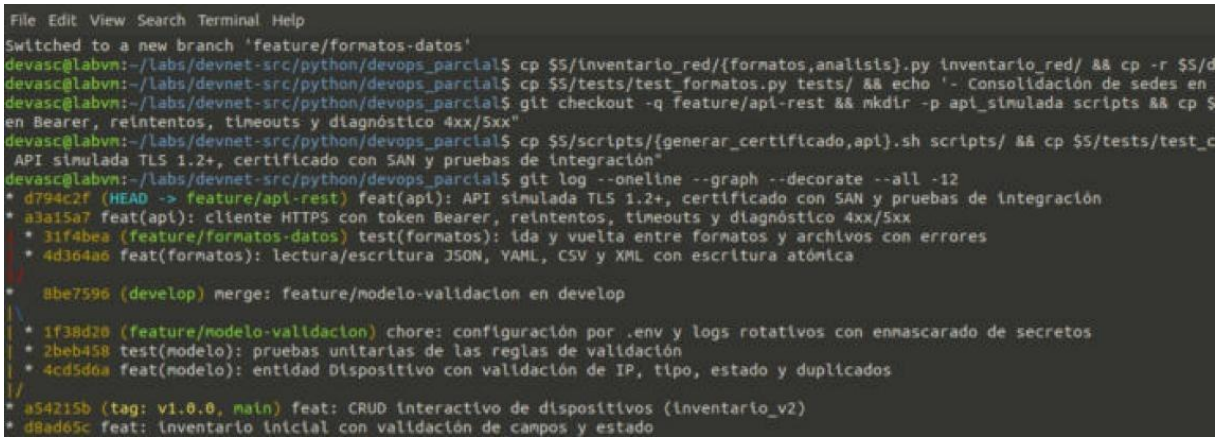


Figura 36. Ramas feature desarrolladas en paralelo.

5.3 Resolución de conflictos

Las ramas feature/formatos-datos y feature/api-rest modificaron la misma línea de README.md. Al integrar la segunda, Git detuvo el merge. Como ambos cambios eran complementarios, conservamos las dos líneas, eliminamos los marcadores, comprobamos con grep que no quedara ninguno y ejecutamos las pruebas antes de confirmar el merge.



Figura 37. Conflicto detectado en README.md.

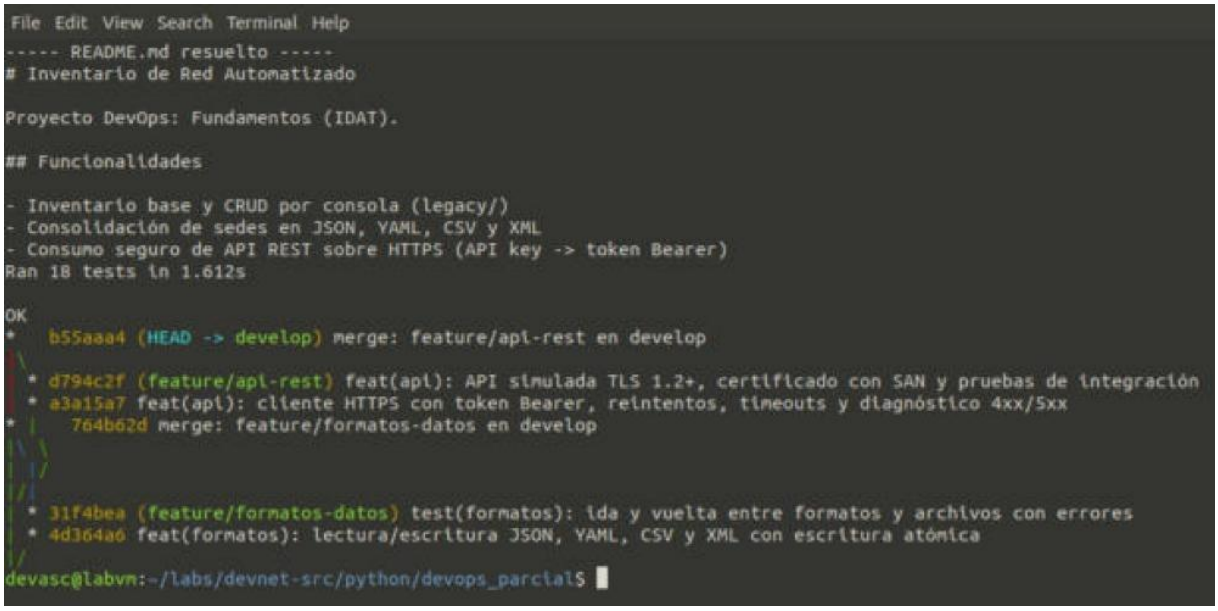


Figura 38. Conflicto resuelto y pruebas superadas.

5.4 Historial final

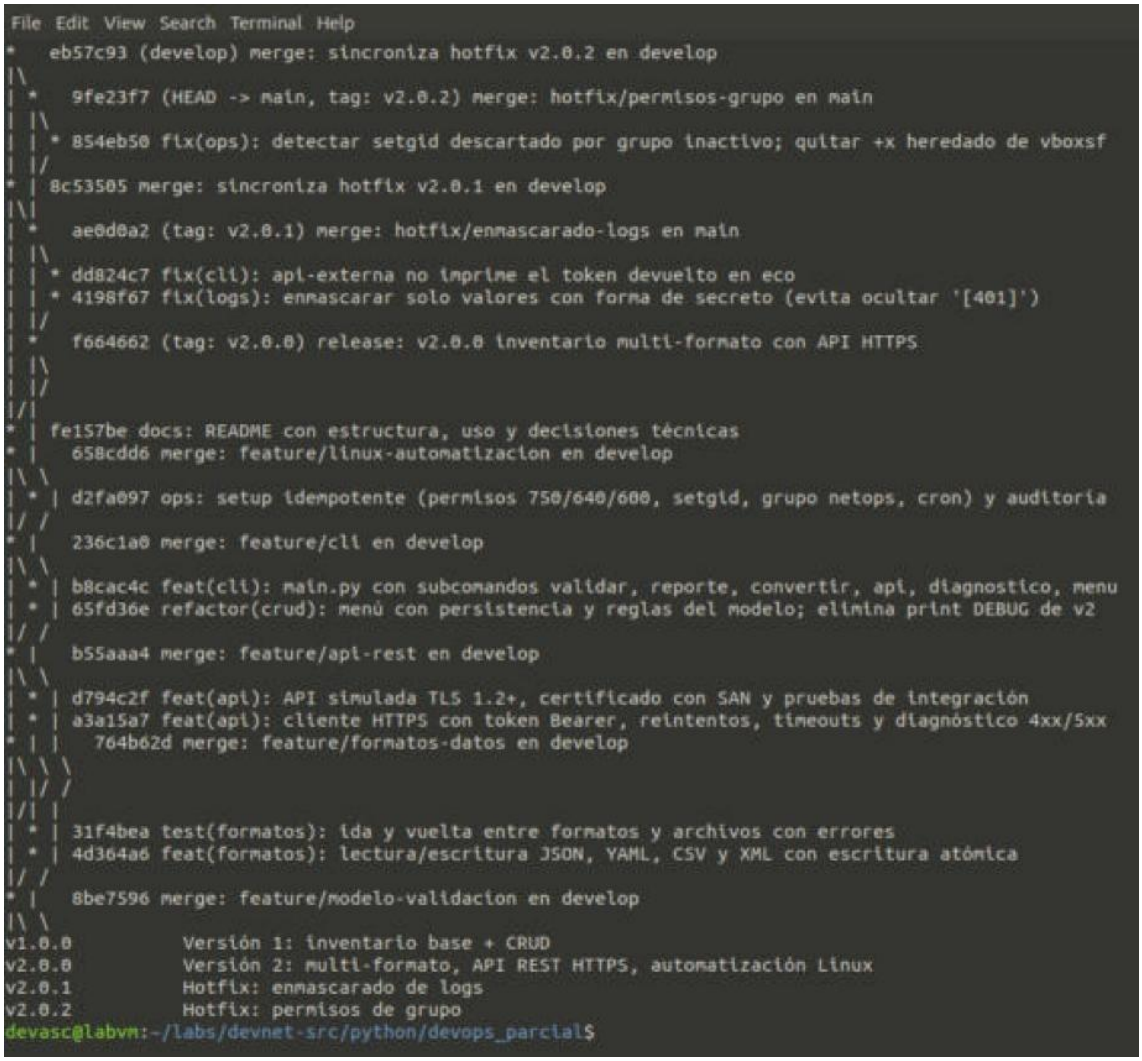


Figura 39. Historial con integraciones, correcciones y versiones etiquetadas.

6. Integración REST API sobre HTTPS

6.1 Servicios integrados

- **API de inventario** (api_simulada/servidor_https.py): servicio HTTPS propio en el puerto 8443 que nos permite reproducir de forma controlada cada tipo de error.
- **httpbin.org**: servicio público real, usado para validar HTTPS con autoridades certificadoras públicas, autenticación Bearer y códigos 401, 404 y 503.

6.2 Autenticación

- **API key → token Bearer:** la API key se envía una sola vez a /auth/token y se recibe un token válido por 5 minutos. Si el token vence, el cliente recibe 401, lo renueva y reintenta una única vez.
- **Secretos fuera del código:** la API key se genera con openssl rand, se guarda en .env con permiso 600 y Git la ignora. El servidor la compara con hmac.compare_digest.
- **TLS verificado:** certificado con SAN (localhost, 127.0.0.1) y TLS 1.2 como mínimo. Nunca usamos verify=False, porque expondría el token a un ataque de intermediario.

6.3 Eficiencia y confiabilidad

- requests.Session reutiliza la conexión TLS entre solicitudes.
- Reintentos con backoff exponencial solo ante 429, 502, 503 y 504, y solo en métodos GET, respetando la cabecera Retry-After.
- Timeouts de 3 s para conectar y 10 s para leer.
- Cada respuesta se valida (tipo de contenido y datos de cada dispositivo).

```
politica = Retry(total=3, backoff_factor=0.5,
    status_forcelist=(429, 502, 503, 504),
    respect_retry_after_header=True,
    allowed_methods={'GET', 'HEAD', 'OPTIONS'})
sesion.mount('https://', HTTPAdapter(max_retries=politica))
r = sesion.request(metodo, url, verify=API_CA_CERT,
    headers={'Authorization': 'Bearer ' + token},
    timeout=(3.05, 10))
```

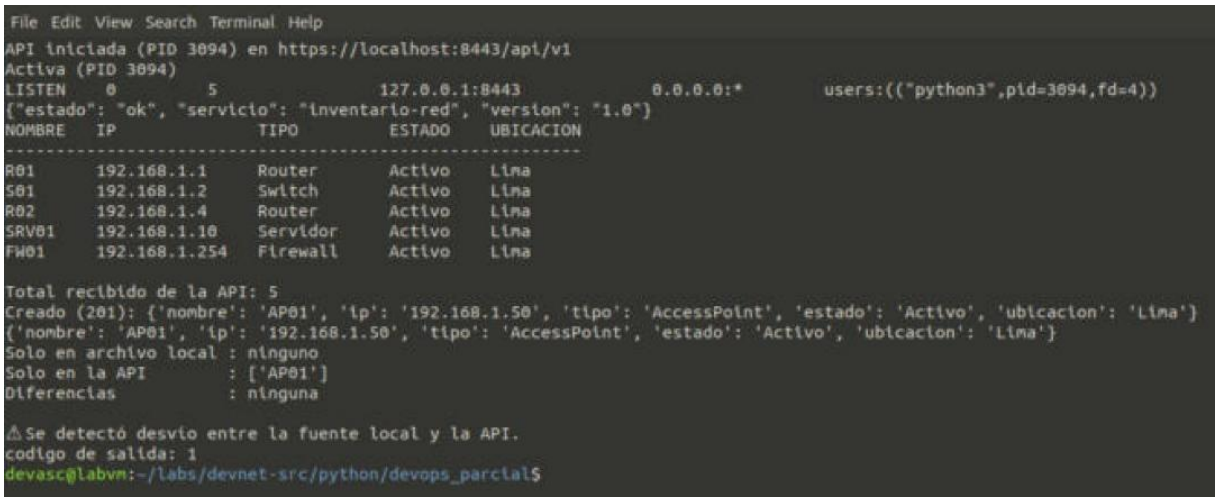


Figura 40. Consulta con token, registro con POST (201) y comparación archivo–API.

6.4 Diagnóstico de errores

El comando ./main.py diagnostico reproduce 15 escenarios y muestra para cada uno la causa probable y la solución, junto con el identificador de solicitud registrado en el log del servidor.

Código	Escenario	Causa	Solución
SSL	Cliente sin la CA del certificado	Certificado no confiable o SAN distinto	Indicar la CA en API_CA_CERT
Conexión	Puerto 8499 en lugar de 8443	Servicio detenido o puerto erróneo	ss -tlnp, curl -v
400	JSON mal formado	Sintaxis inválida	Enviar con json= de requests
401	Sin token o token vencido	Falta autenticación	Renovar el token
403	API key incorrecta	Credencial inválida	Revisar .env y permisos
404	Dispositivo inexistente	Recurso o ruta incorrecta	Verificar /api/v1 y el nombre
405	DELETE no soportado	Método no permitido	Usar GET o POST
409	Registro duplicado	El recurso ya existe	Consultar antes de crear
415	Sin Content-Type JSON	Tipo de contenido no soportado	Enviar application/json
422	IP 300.1.1.1 y estado «Roto»	No cumple las reglas	Corregir los campos indicados
429	Límite de solicitudes	Rate limit	Esperar Retry-After
500	Error del servidor	Fallo interno	Reportar con el identificador
503	Mantenimiento	Servicio no disponible	Reintentar con backoff
Timeout	Respuesta de 4 s con límite de 1,5 s	Servicio lento	Ajustar el timeout

Tabla 7. Errores diagnosticados.

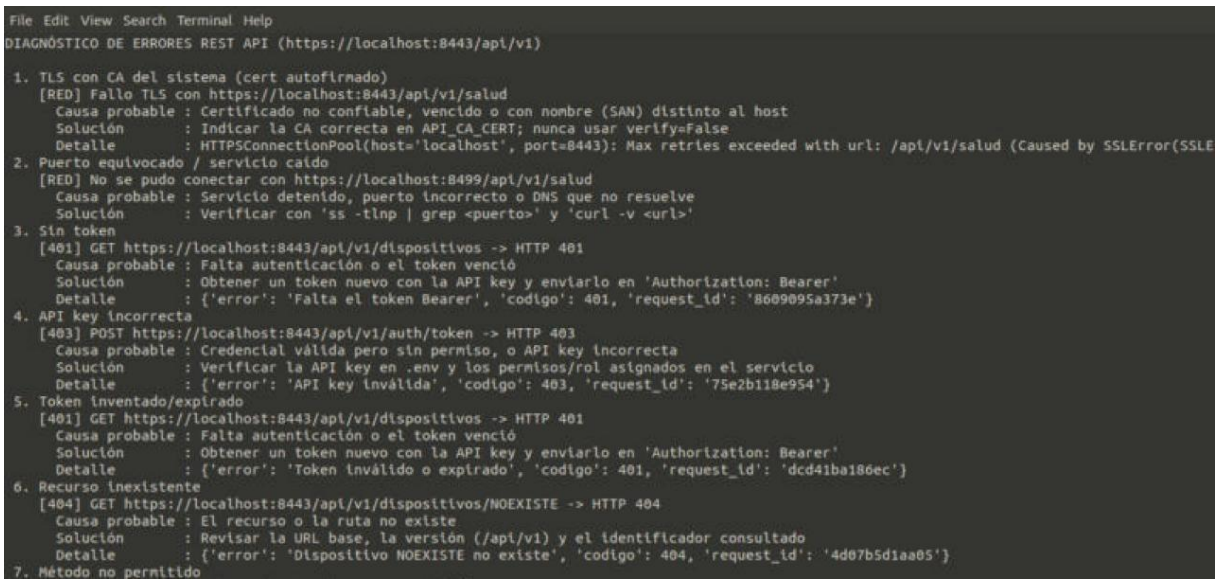


Figura 41. Diagnóstico de errores (parte 1).

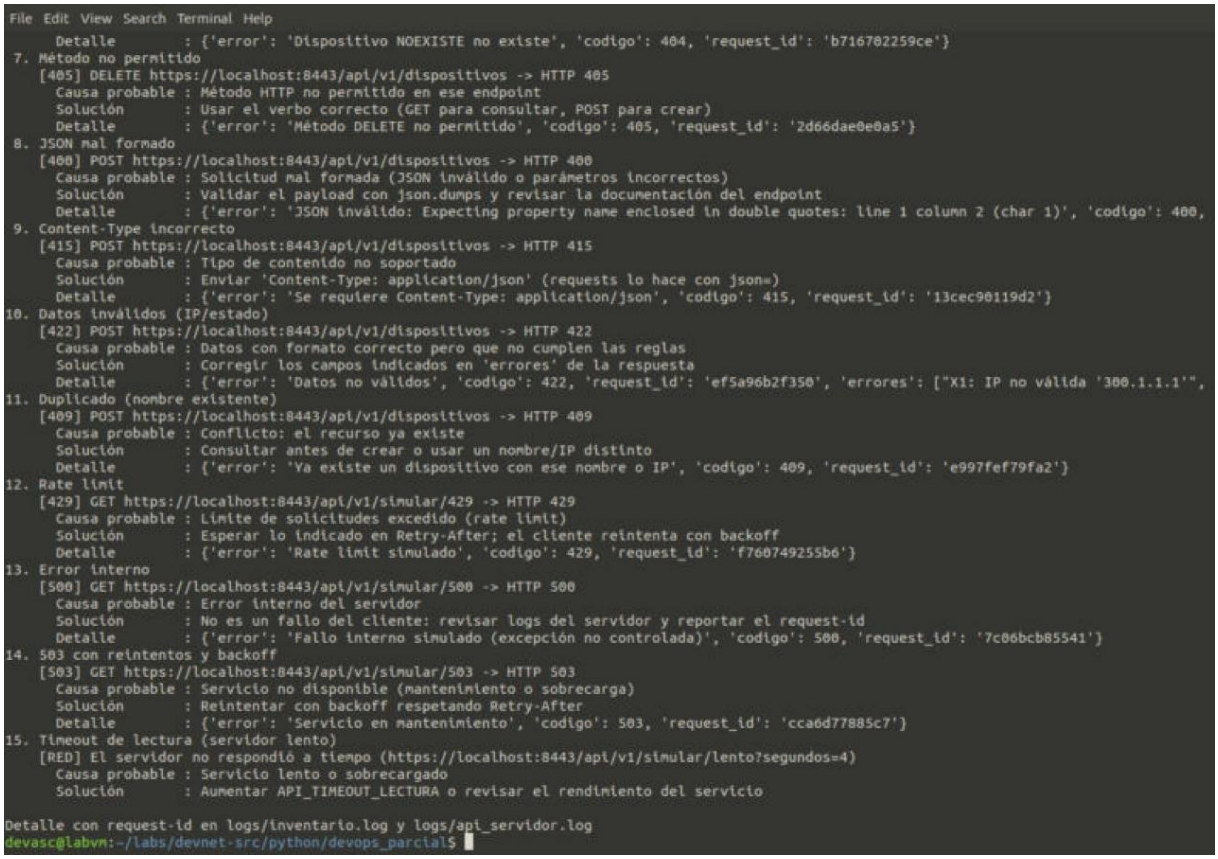


Figura 42. Diagnóstico de errores (parte 2).

7. Problemas encontrados y soluciones

Además de los errores reproducidos a propósito, durante el despliegue encontramos los siguientes problemas:

Problema	Diagnóstico	Solución
La máquina virtual respondía muy lento	VirtualBox indicaba «Execution Engine: native API»: se ejecutaba sobre Hyper-V, sin paginación anidada	Desactivamos Hyper-V y la integridad de memoria en Windows; asignamos 4 vCPU y 8 GB. La VM pasó a usar VT-x
logs/ y reportes/ perdían el bit setgid	Linux descarta el setgid si quien ejecuta chmod no es root y aún no tiene activo el grupo (usermod -aG requiere una nueva sesión)	El script ahora lo detecta y avisa; se aplica con sudo o tras iniciar sesión de nuevo (v2.0.2)
Un log mostraba «Sin token: **»**	El filtro de secretos ocultaba cualquier texto después de «token:»	Solo se ocultan valores con forma de secreto; se añadió una prueba (v2.0.1)
Se imprimía el token de prueba	httpbin.org devuelve el token en su respuesta	Los campos «token» nunca se muestran (v2.0.1)
Un archivo se versionó como ejecutable	Los archivos copiados desde la carpeta compartida de VirtualBox llegan con permiso 770	Permiso corregido y normalizado en Git (v2.0.2)

Tabla 8. Problemas y soluciones.

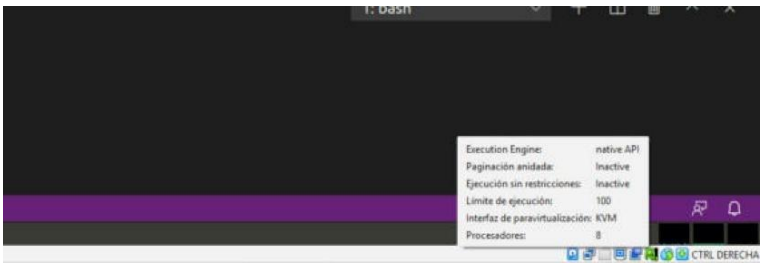


Figura 43. Motor de ejecución «native API» antes de la corrección.

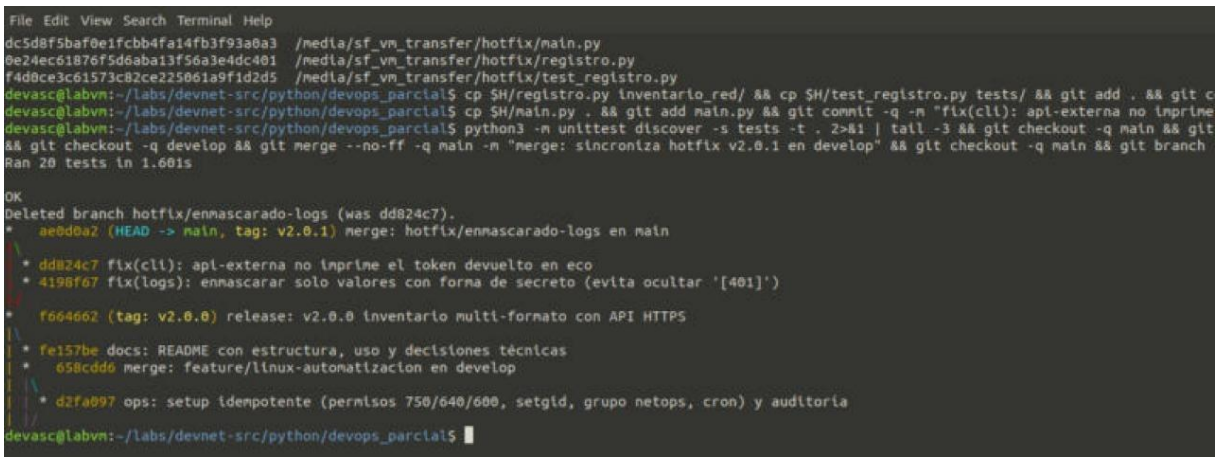


Figura 44. Corrección v2.0.1 integrada en main con 20 pruebas superadas.

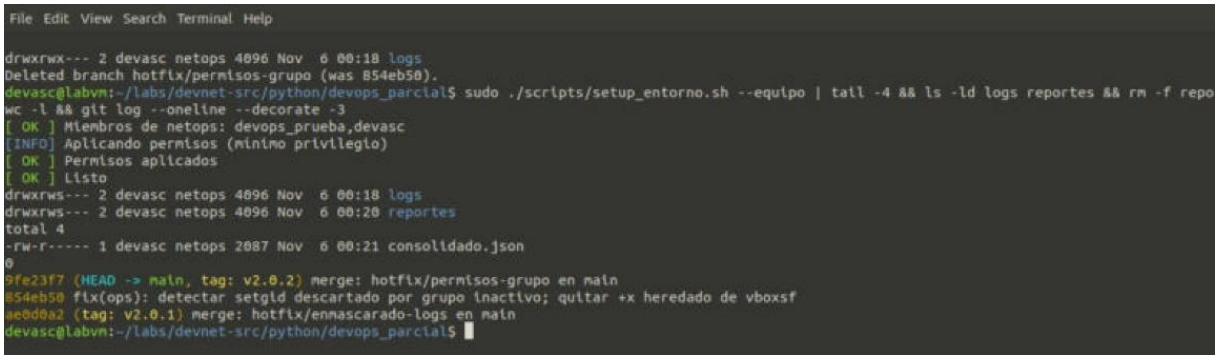


Figura 45. Corrección v2.0.2: setgid restaurado y reportes del grupo netops.

8. Sustentación de decisiones técnicas

Decisión	Alternativas	Motivo	Impacto
requests + Session + Retry	urllib, httpx	Incluido en la VM; reutiliza la conexión TLS y ofrece backoff sin código propio	Menor latencia y tolerancia a fallos
API key → token Bearer	Basic Auth, API key en cada solicitud, OAuth2 completo	Sigue el patrón de credenciales de cliente de OAuth2 sin depender de un proveedor externo	Menor exposición de credenciales
API propia con http.server y ssl	Flask, simulaciones con mocks	Sin dependencias; permite forzar cada código HTTP y probar TLS real	Pruebas reproducibles sin Internet
Certificado con SAN	verify=False	Los clientes validan el SAN; desactivar TLS anula la seguridad	Confidencialidad e integridad
yaml.safe_load y ElementTree	yaml.load, lxml	safe_load no ejecuta objetos arbitrarios; ElementTree es estándar	Seguridad sin dependencias
unittest	pytest	Incluido en Python	Pruebas portables
Flujo main/develop/feature/hotfix	Una sola rama	Separa lo estable de lo que se integra	Trazabilidad y reversión simple
Bash idempotente	Comandos manuales, Ansible	Disponible en cualquier Linux y repetible	Despliegue rápido y sin errores manuales
Grupo, setgid y cuenta de servicio	Permisos 777, usuario compartido	Mínimo privilegio y auditoría por usuario	Escalable a más integrantes

Tabla 9. Decisiones técnicas.

9. Conclusiones

- El proyecto pasó de un script con datos fijos a una herramienta que consolida los inventarios de cuatro sedes en cuatro formatos, los valida, los compara con una API HTTPS autenticada y genera reportes programados.
- La seguridad se aplicó en capas: permisos mínimos, secretos fuera del repositorio y de los logs, TLS verificado y tokens de corta duración.
- Los errores de la API se traducen en una causa y una solución concretas, con un identificador para rastrearlos en el servidor.
- El historial de Git documenta la evolución del proyecto, incluidos un conflicto resuelto y dos correcciones surgidas durante el despliegue.

Anexo: guía de ejecución

Desde la carpeta del proyecto en la máquina virtual:

```
./scripts/setup_entorno.sh
sudo ./scripts/setup_entorno.sh --equipo
./scripts/api.sh iniciar
python3 -m unittest discover -s tests -t .
./main.py reporte datos/ --exportar reportes/inventario.json
./main.py validar datos/pruebas/dispositivos_con_errores.csv
./main.py api listar --estado Activo
./main.py diagnostico
./main.py api-externa
./main.py menu
```